

## 0.29.4. La fonction `un_pas()` — un seul pas, sans attendre

`#include <un_pas>` : `un_pas(x, y, tuile, sensX, sensY)` fait faire UN pas à la tuile, tout de suite, sans bloquer le jeu.

### 1. Ce qu'il faut comprendre

**Ce qu'elle fait :** `un_pas(x, y, tuile, sensX, sensY)` fait faire **un seul pas**, tout de suite, sans attendre : 1 vers la droite ou le bas, -1 vers la gauche ou le haut, 0 immobile. Elle ne bloque pas : on l'appelle dans la boucle, et d'autres choses bougent en même temps.

**La ligne à écrire :** `#include <un_pas>`. Ici aussi `<chaque>`, pour rythmer les pas.

**Ses arguments :** la place, la tuile, le sens en x et en y. Seule sur sa ligne, elle range la nouvelle place dans x et y.

**Ce qu'elle coûte :** environ **180 octets**, plus la police entière.

**Essaie :** `un_pas(x, y, ALPHABET[0], 1, 1);` : en diagonale.

#### On retrouve `un_pas` dans 15 leçons

[0.30](#), [0.30.1](#), [0.30.2](#), [0.31](#), [0.31.1](#), [0.31.2](#), [0.76.2](#), [0.76.3](#), [0.76.4](#), [0.76.5](#), [0.76.6](#), [0.79](#), [0.79.1](#), [0.84](#), [0.85](#)

### 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1182 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

|                   |       |                              |                                                     |
|-------------------|-------|------------------------------|-----------------------------------------------------|
| démarrage         | 63 o  | ← la console, à l'allumage   | – la mise en route de la console                    |
| chaque_minuteur() | 124 o | ← main()                     | – une fonction fournie par le langage               |
| un_pas()          | 117 o | ← main()                     | – une fonction fournie par le langage               |
| main()            | 56 o  | ← démarrage                  | – ta fonction                                       |
| VBlank            | 8 o   | ← la console, à chaque image | – compte les images, 60 fois par seconde            |
| AttendreImage     | 28 o  | ← image()                    | – attend l'image suivante                           |
| AttendreVBlank    | 16 o  | ← démarrage                  | – attend que l'écran soit libre                     |
| AttendreAcces     | 12 o  | ← EffacerCases               | – attend de pouvoir écrire une case                 |
| AdresseCase       | 10 o  | ← effacer(), poser()         | – trouve l'adresse d'une case de l'écran            |
| EffacerCases      | 12 o  | ← effacer()                  | – efface des cases                                  |
| CopierTuiles      | 18 o  | ← démarrage                  | – copie les lettres et les dessins en mémoire vidéo |
| EffacerCarte      | 14 o  | ← démarrage                  | – vide les deux cartes de l'écran                   |
| Tuiles            | 704 o | ← CopierTuiles               | – la police entière                                 |

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

|           |     |                                                                         |
|-----------|-----|-------------------------------------------------------------------------|
| effacer() | x 1 | 15 o sur place, dans un_pas() + AdresseCase (10 o), EffacerCases (12 o) |
| poser()   | x 1 | 13 o sur place, dans un_pas() + AdresseCase (10 o)                      |
| image()   | x 1 | 3 o sur place, dans main() + AttendreImage (28 o)                       |

```
images()          x 1      2 o sur place, dans chaque_minuteur()
Tes fonctions appelées :
un_pas()          x 1      22 o sur place, dans main()
chaque_minuteur() x 1      10 o sur place, dans main()
```

### 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <un_pas> : un seul pas, sans attendre ----
// Toutes les 250 ms, le A fait un pas vers la droite, tant qu'il n'est pas au bord.

#include <un_pas>      // un_pas() : un pas, tout de suite
#include <chaque>
#include <ALPHABET>
```

### 2. UNE VARIABLE — X

```
uint8_t x = 0;
```

### 3. UNE VARIABLE — Y

```
uint8_t y = 8;
```

### 4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

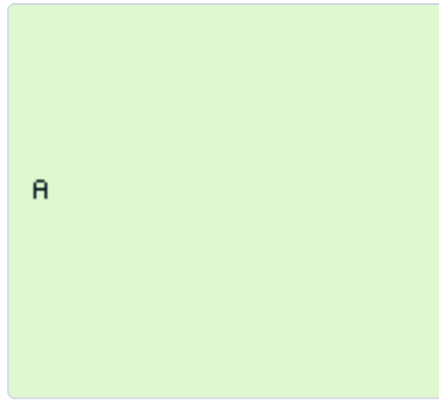
```
int main() {
    while (true) {
        image();
        if (chaque(250) && x < 19) {
            un_pas(x, y, ALPHABET[0], 1, 0);    // un pas à droite ; x grandit de 1
        }
    }
}
```

## 3. Ce qui se passe, étape par étape

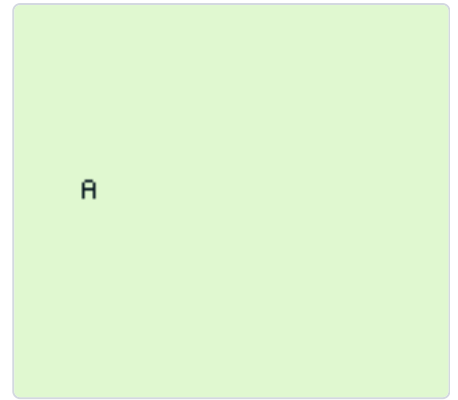
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



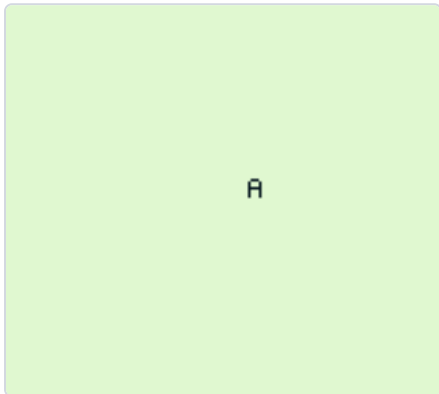
la toute première image — la console vient de démarrer — image 1



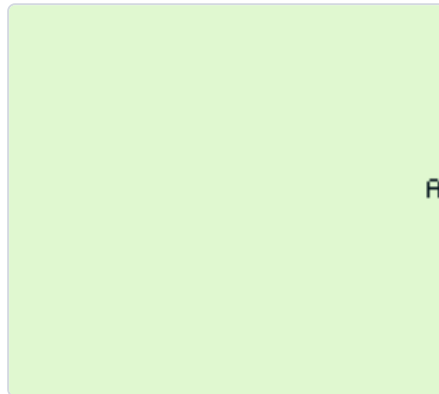
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

## 4. Ce qu'on doit voir

Un A qui avance d'une case tous les quarts de seconde, jusqu'au bord droit.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A avance case par case (9 places)

✓ vers la droite, sur la ligne 8

Ensuite : **0.30. Un pas sans attendre : un\_pas** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++